DOCUMENT AI · PROJECT UPDATE · NOVEMBER 2026

Model evaluation results and a proposed solution

What I tested, which models I recommend, what our current server can run, how the pipeline would work, and what we need from stakeholders before we build.

We can build this on the L4 we already have, but stakeholders still have to define "accurate enough".

Fits the L4

Every recommended model fits on one 24 GB GPU. They load one at a time, in batches. No hardware purchase is needed to start.

One model per job

Small specialist models beat the 32B general model at reading tables and pages, and run up to 15 times faster.

99% is a system

No model reached 99% of numbers. High accuracy comes from cross-checks and targeted human review, not from a bigger model.

What comes next: answers from stakeholders on the current process, acceptable error and how they want to use the data; a labelled sample of 100–200 documents; and approval to pilot.

The starting prototype proved the idea. Nothing had been measured yet.

What it did

- Read PDF text and tables directly.
- Ran classic OCR on scanned pages.
- Retrieved excerpts and asked one hosted general model a fixed set of questions.
- Wrote the answers to a spreadsheet.

What was still open

- Which models read tables, pages, charts and clauses best?
- How accurate is each one, especially on numbers?
- What hardware does each need?
- How should results be checked before anyone relies on them?

> So the first job was a measured evaluation, before changing the design.

A controlled test, built so the result can be trusted.

11
open-weight models we could host ourselves

7
document jobs: tables, pages, charts, text regions, layout, contracts, extraction

3
rounds: explore, then confirm the leaders on data the models had never seen

0
scores decided by another AI. Code checked every answer against an answer key

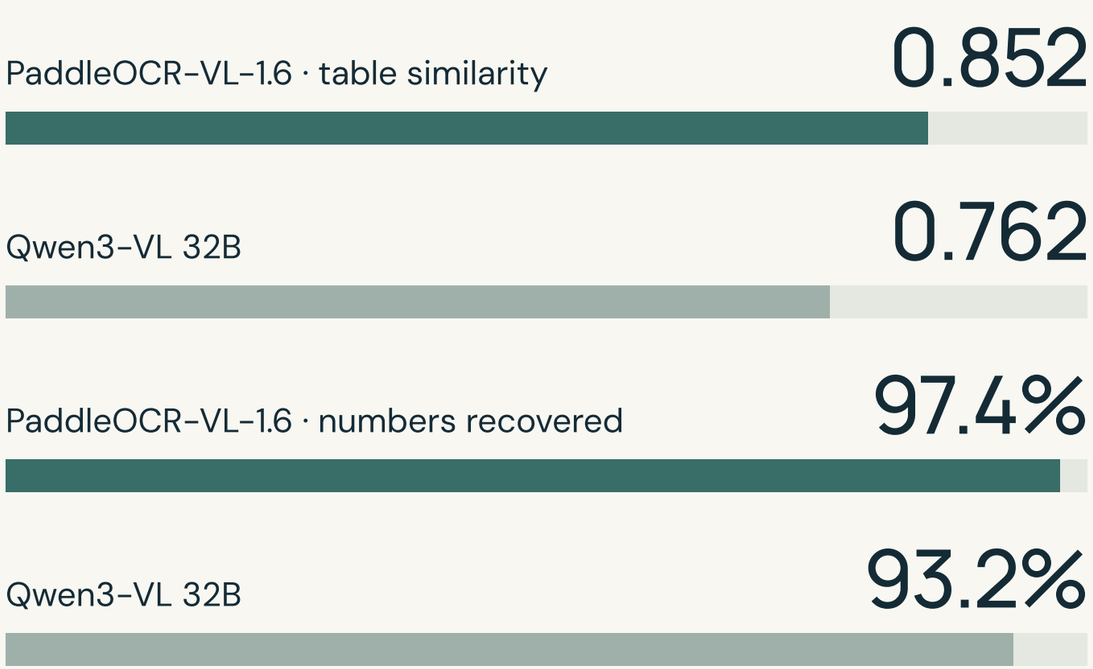
- Public benchmark data only. No work documents left work storage.
- Pass rules were written down before scoring, and failures counted as zero.
- Comparisons were paired, with a statistical test and a correction for running several tests.

One model per job.

JOB IN THE PIPELINE	MODEL	WHY	STATUS
Budget and pricing tables	**PaddleOCR-VL-1.6** (0.9B)	Beat the 32B on 100 unseen financial tables; kept 97.4% of numbers	Confirmed
Full-page text	**Nemotron 3.5 Parse** (0.63B)	Most accurate page text and about 15× faster than the 32B	With number check
Text regions and charts	**Qwen3-VL 8B**	Matched the 32B on text; chart method chosen per task	Keep
Finding tables and figures	**DocLayout-YOLO**	Figure F1 of 0.876, against 0.612 for the parsers	Keep
Field and clause extraction	**gpt-oss-20b** (fits the L4) or **Qwen3 32B** (needs an L40S)	Best on contract questions; both beat the hosted Nemotron setup	Reviewer required

All open-weight; all can be stored on our server and loaded when needed

A 0.9B table reader beat the 32B model.



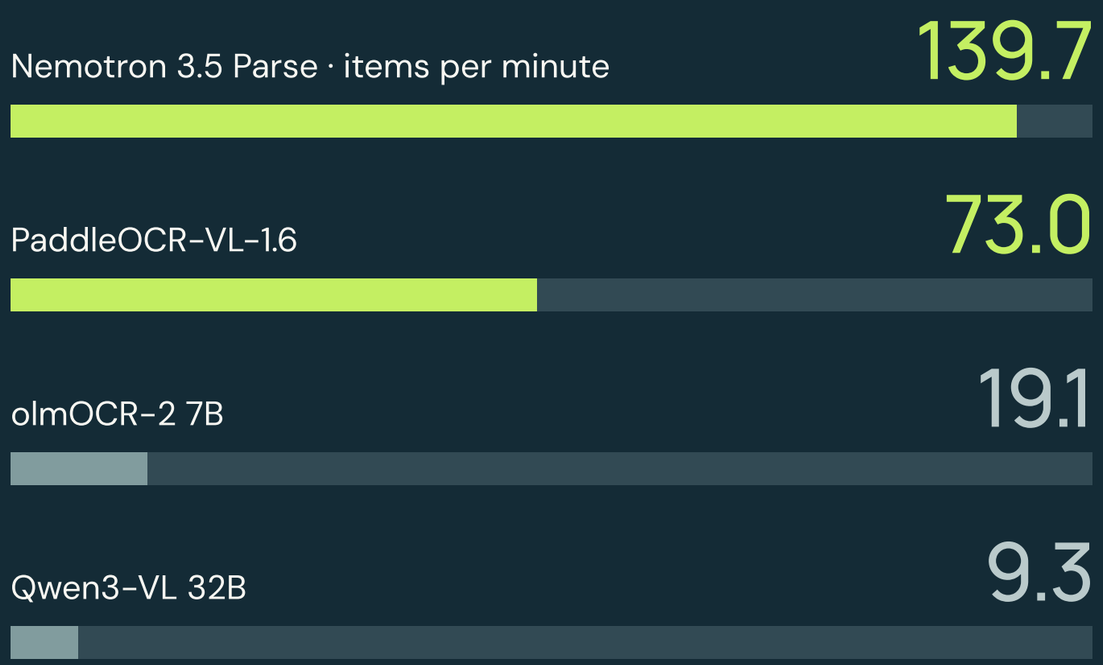
100 financial-report tables the models had never seen, with the same output limit for both. The pass rule was written down before scoring, and the model passed.

Why it matters: budgets and pricing tables are where the value is, and they are what breaks first.

Caveat: 97.4% is not 99%. Amounts still go to a reviewer.

Pre-registered confirmation; paired bootstrap with Holm correction

Small parsers read pages better and far faster.



Page text: Nemotron Parse scored 0.899 word overlap, against 0.820 for the 32B, on 80 unseen pages.

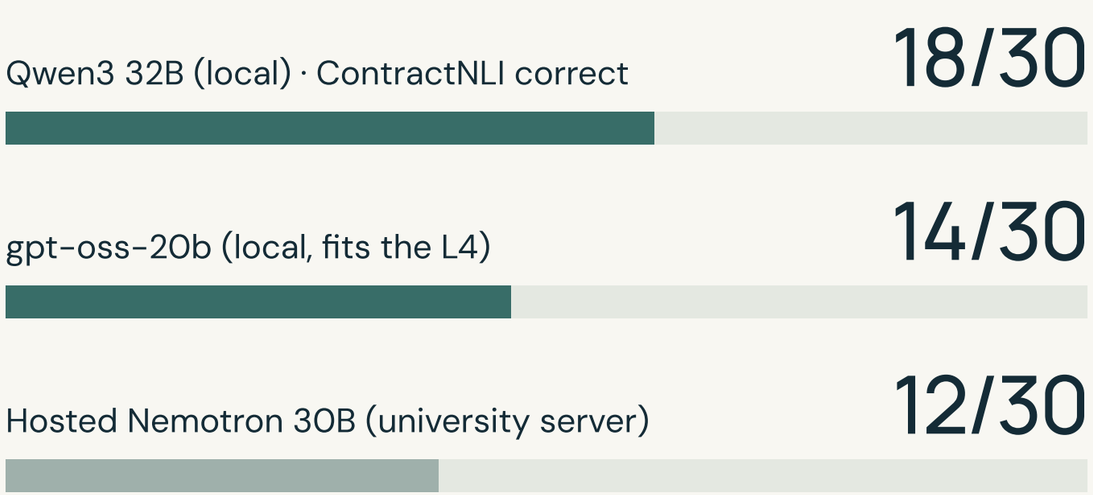
Numbers: 88.2% against 88.3%. That is a tie on average, but it is not yet proven safe, so a numeric check runs on its output.

ESTIMATED PAGES PER HOUR ON THE TARGET SERVER

MODEL	L40S (48 GB)	L4 (24 GB)
Nemotron Parse	4,000–6,000	1,500–2,500
PaddleOCR-VL-1.6	2,000–3,000	800–1,400
Qwen3-VL 32B	250–400	Does not fit well

Bars: measured relative speed, same GPU for all models. Table: extrapolated from GPU memory bandwidth and compute; the pilot measures real rates

Extraction is the weakest link, and it is why a reviewer stays.



Same 60 public contracts for every model. Local models beat the hosted service.

On CUAD, simply answering "absent" every time scores 15/30, and no model beat that. Contract reasoning is still unsolved at this model size.

This is the step where a larger model (on HPC) or a frontier API might help. That is worth testing, not assuming.

60 public CUAD and ContractNLI documents; scored by code against answer keys

Yes. Each model fits the L4 on its own, so we load them one at a time.

MODEL	JOB	APPROX. GPU MEMORY	L4 · 24 GB	L40S · 48 GB
DocLayout-YOLO	Find regions	< 1 GB	**Fits**	**Fits**
Nemotron 3.5 Parse (0.63B)	Page text	2–4 GB	**Fits**	**Fits**
PaddleOCR-VL-1.6 (0.9B)	Tables	3–5 GB	**Fits**	**Fits**
Qwen3-VL 8B (FP8)	Crops, charts	10–14 GB	**Fits**	**Fits**
gpt-oss-20b	Extraction	14–18 GB	**Fits with the whole card**	**Fits**
Qwen3 32B (4-bit)	Extraction, stronger	22–28 GB	**Does not fit well**	**Fits**

If the GPU is shared with other services, the larger steps run in scheduled batch windows when the card is free. The small readers can run alongside other services.

Memory = weights plus runtime headroom; confirm with a load test on the server. Planning prices: L4 server $6k–10k, L40S server $16k–21k (dated estimates, not quotes)

Identify what each part of a document is, then send it to the model that reads that kind best.

1
Ingest
Register each file: fingerprint, version, source, policy check

2
Identify
Born-digital or scanned? Find the regions on each page

3
Read in batches
Each region type goes to its own model (below)

4
Extract
A text model picks field values and the evidence behind each

5
Verify
Code checks totals, dates, IDs and the copied evidence

6
Review
A person checks only flagged values

7
Deliver
Database, review screen, exports, search

Text
Born-digital: read directly, no model.
Scanned: Nemotron Parse

Tables
PaddleOCR-VL-1.6

Charts and images
Qwen3-VL 8B

Small text regions
Qwen3-VL 8B, with a second reader on numbers

Every value keeps a link to its page and region, so a reviewer, or anyone downstream, can see exactly where it came from.

Queue everything by type, then run one model at a time over its whole queue.

1 · Queue

New documents are split into regions, and each region is queued by type: text, table, chart, crop.

2 · Load and run

Load one model, process its entire queue, write the results, then unload it.

3 · Next model

Layout, tables, pages, crops, extraction, then verification on CPU.

4 · Repeat

Run every batch window, nightly or hourly, until the queue is empty. Jobs resume after any interruption.

~1 min

to swap models, spread over thousands of items in each batch

2–3 days

estimated for 100,000 scanned pages on an L4 (about 1 day on an L40S)

0

models competing for GPU memory, because only one is loaded at a time

Throughput figures are estimates until the pilot measures them on the server

No model is 99% on its own. The system can get there.

Built-in checks

- **Native text first.** Numbers in born-digital PDFs are exact.
- **Two readers on every number.** If they agree, accept; if they disagree, send it to review.
- **Code checks.** Rows add up to totals, dates fall in range, and IDs match existing records.
- **Evidence copy check.** A changed amount can never pass as "verified".

Flag hard documents up front

- Score each document's risk from scan quality, table density, reader disagreement and model confidence.
- Learn which document types usually fail. New documents that look like them go straight to review.
- The review queue then targets the hard cases instead of a random sample.

The target becomes "X% of delivered values correct, at a known review rate". Stakeholders choose X, and the pilot tells them what review rate it costs.

A proposal for stakeholders: set the bar by how much each field matters.

TIER	EXAMPLES	DELIVERED ERROR	HOW IT IS MET
Critical	Amounts, totals, rates, dates, periods, IDs	≤ 0.5%	Native text or two readers agreeing, plus code checks; every disagreement reviewed
Important	Parties, terms, reporting rules, approvals	≤ 2–5%	Evidence check, review below a confidence threshold, 5–10% sampled QA
Descriptive	Scope, summaries, themes	≤ 10%	Shown with its evidence and labelled as interpretation; spot-checked

99%
numbers recovered: the original plan's bar

97.4%
best measured: tables

88%
numbers on full pages

60%
best contract-statement score

Original quality gates: ≤1% character error, ≥99% numbers, ≥90% field accuracy, ≤2% failures, all pending domain-owner sign-off

Ten sample documents cannot tell us if this works. We need 100–200.

≈300

checked values with zero errors are needed just to show the error rate is below 1%, with 95% confidence. For 0.5%, it is about 600.

- **Sample 100–200 documents**, spread across document types, sources, years and scan quality. That gives 2,000+ field values.
- **Label them once**, with staff who know the documents. Split them so we tune on one part and test on the other.
- **Shortcut:** if staff already key these values into a system or spreadsheet, those historical entries are ready-made answer keys.
- The same labelled set later supports fine-tuning a model on our own templates.

"Rule of three": zero errors in n checks bounds the true error rate at about 3/n with 95% confidence

An escalation ladder: test each step before paying for it.

1	START HERE **Our server: L4, or an L40S if needed** Specialist models, checks and review. Data stays on our hardware.	COST Hardware we already own. An L40S server is about $16k–21k if the 32B extractor is needed.	LIMIT Contract reasoning tops out at the 32B model.
2	TEST ON PUBLIC DATA **University HPC: much larger open models** Muse Spark Open (816B), Kimi K3 (2.8T), Qwen3.5-397B and others, on shared H200 nodes.	COST An allocation, not a purchase. A test run is about 250–500 GPU-hours.	LIMIT Eligibility and data classification still need confirming.
3	ONLY WHERE POLICY ALLOWS **Frontier cloud API: Azure OpenAI, Gemini** Used for the extraction and reasoning step. The specialist readers stay in place.	COST About $3–13 per 1,000 pages. An evaluation run is under $10 per model.	LIMIT Data leaves our infrastructure. Frontier models still read tables worse than PaddleOCR-VL.

Each step uses the same tests and pass rules as this evaluation, so results are comparable

Cloud API fees are small at our volumes. Data policy, not cost, is the real constraint.

MODEL (LIST PRICE PER MILLION TOKENS, IN / OUT)	PER 1,000 PAGES	EVALUATION ON 500 LABELLED PAGES	THE WHOLE HISTORICAL BACKFILL (≈1.9M PAGES)
Gemini 3 Flash ($0.50 / $3)	≈ $3	≈ $2	≈ $6k
GPT-6.1 Sol on Azure ($2 / $10)	≈ $11	≈ $6	≈ $21k
Gemini 3.1 Pro ($2 / $12)	≈ $13	≈ $7	≈ $24k

Where "$6k–25k" came from

It is the API fee to read every page of a 125,000-attachment historical backlog (about 1.9M pages). It is not a hardware price. Reading a smaller current workload, such as 10,000 pages, would cost about $30–130.

What to budget for an API test

Less than $50 in fees covers several providers on the labelled sample. The real costs are staff time and getting data approval for the provider.

Assumes about 1,500 input and 800 output tokens per page; excludes reasoning tokens and retries. List prices October 2026; verify before budgeting

Two architectures. The same pipeline; the difference is where the models run.

A · Open-weight back end on our server (recommended start)

- Processing runs on the L4 in batch windows.
- Results go to a database, each value with its evidence.
- Users get a web app for review, search, filtering and export. It can be hosted in Azure or internally, calling the server's results.
- Escalation goes to university HPC or an API only if needed.

B · Azure-hosted processing

- Azure OpenAI for extraction, plus Azure's document OCR service or our specialist models in a container.
- Storage and the web app also live in Azure.
- Only for data approved for Azure. Per-page fees apply, and table reading may be weaker.

Open question: what users actually need on the front end, whether that is tables, filters, search, natural-language questions, dashboards or exports. That decides the web app, not the models.

Requirements to confirm with stakeholders before building.

Today's process and the gap

- What process does this replace? Who does it today, and how many hours does each document take?
- What is the gap: speed, backlog, cost, errors, missed terms, or no visibility at all?
- What decisions or reports depend on this data?
- Is the goal mainly saving staff hours, or doing something not possible today?

Accuracy and review

- Which fields are critical, and what error rate is acceptable for each?
- What happens today when a value is wrong, and who catches it?
- How much reviewer time is available each week?
- If self-hosted models fall short, is a cloud API or university HPC acceptable for this data?

A short working session with stakeholders and real documents answers most of these fastest

What we ingest, and how people want to use the results.

The data

- Which document types, and how many: documents, pages, share scanned, file formats?
- How many new documents arrive each week? Is there a deadline for the backlog?
- Is there a sample where the values are already extracted, such as a spreadsheet or system records?
- What is the data classification? Is cloud processing, or university HPC, approved for it?

How they will use it

- How do they look at this information today?
- Do they want tables and exports, filters, keyword search, natural-language questions, dashboards, or alerts such as deadlines?
- Where should results land: a spreadsheet, a database, the system of record, a dashboard, a web app?
- Who will use it, and how often? Must every value link to its source page?

Operational questions that decide whether this runs reliably.

Running it

- **Ownership:** who operates the pipeline, updates models and handles failures?
- **GPU schedule:** agree batch windows with any other services on the GPU.
- **Access control:** who may see which documents and results?
- **Model licences:** confirm each one is cleared for university use.

Proving it was worth it

- **Baseline:** time and cost per document today, measured before launch.
- **Success metric:** accepted values per reviewer hour, plus delivered error rate.
- **Integration:** the system of record, and who owns that connection.
- **Rollout:** who trains the reviewers, and how feedback improves the routing.

Twelve weeks from approval to a measured answer.

WHEN	WORK	RESULT
Weeks 1–2	Stakeholder questions; data access and classification; collect the 100–200 sample documents and any existing records	Requirements, acceptance bars, sample set
Weeks 2–6	Build the pipeline on the L4: ingest, identify, batch reading, extraction, checks, review queue; label the sample	A working pipeline on real documents
Weeks 6–8	Measure accuracy and review rate per field tier; map which document types fail	"Can our server meet the bar?"
Weeks 8–12	Only if needed: test HPC-hosted large models and an API on the extraction step; backlog throughput pilot	The final architecture and cost recommendation

No hardware purchase or API spend needed before week 8

Four steps to move forward.

1. **Build the pipeline on the existing L4.** No purchase now.
2. **Get the stakeholder questions answered**, ideally in one working session with real documents.
3. **Collect a sample of 100–200 documents**, plus any existing records of their values, and a data-classification ruling.
4. **Line up the fallbacks:** university HPC access and a small API cost test, used only if the pilot falls short.